IESB • CONSTRUÇÃO DE BACKEND

Projeto 2 — Sistema de Biblioteca

Guia passo a passo do aluno (Django ORM avançado)

Disciplina: Construção de Backend (Django / Python)
Professor: Prof. Me. Bruno Assunção Dias
Tipo: Projeto de portfólio — publicar no GitHub
Duração estimada: 8 a 12 horas, divididas em 4 sessões
Portal da disciplina: devquestrpg.vercel.app/projetos/projeto-2

1. O que você vai construir

No Projeto 1 você trabalhou com **um** Model e um CRUD simples. Agora o desafio é maior: um **sistema de biblioteca** com **quatro Models conectados**, em que as regras (quem pode pegar qual livro, e quando um empréstimo está atrasado) ficam no **código Python**, e não apenas no banco.

Ao final, seu sistema vai:

- Cadastrar **Autores, Livros e Membros** pelo Django Admin.
- **Bloquear** o empréstimo de um livro sem cópias disponíveis.
- Ao emprestar: definir o prazo de **14 dias** e **diminuir** `copias_disponiveis`.
- Ao devolver: registrar a data, marcar como devolvido e **devolver a cópia ao estoque**.
- Listar os empréstimos de um Membro, **destacando os atrasados**.
- Gerar o relatório dos **livros mais emprestados** usando `annotate()` e `Count()`.

Conceitos que você vai praticar

Conceito	Onde aparece
`ForeignKey` (um-para-muitos)	Autor → Livro, Livro → Emprestimo, Membro → Emprestimo
`related_name`	`autor.livros.all()` em vez de uma consulta manual
Regra de negócio na View	Verificar cópias antes de emprestar
`@property` no Model	`esta_atrasado` — dado calculado, sem coluna no banco
Agregação no ORM	`annotate()` + `Count()` no relatório
Admin customizado	`list_display` e `TabularInline`

> **Combinado de trabalho**
>
> Cada passo termina com um **Checkpoint**. Só avance quando o checkpoint funcionar. Se algo quebrar, o erro quase sempre está no passo imediatamente anterior.
>
> Os blocos de código são **arquivos completos**, com o caminho no título, para copiar e colar. Leia o código antes de colar: o objetivo é entender, não só copiar.

Como ler as cores do código

Os blocos de código usam as mesmas cores do VS Code (tema Dark+). Aprender a reconhecê-las ajuda a ler e a **achar erros de digitação**: se uma palavra está com a cor errada, provavelmente foi escrita errada.

```
from / def / if / return   Palavras-chave da linguagem (roxo)
emprestar_livro(           Nomes de funções e métodos (amarelo)
Livro  Autor  Emprestimo   Classes e Models (verde-água)
'ATIVO'  "Emprestimo"      Textos entre aspas (laranja)
livro  membro  request     Variáveis e atributos (azul-claro)
True  None  self           Valores especiais e self (azul)
14  3                      Números (verde-claro)
# comentário               Comentários: explicam o código e não são executados (verde)
{% if %}  {{ nome }}       Delimitadores das tags e variáveis do template Django (roxo)
```

2. Estrutura de pastas do projeto

Esta é a estrutura final. Use-a como mapa: ao criar cada arquivo, confira se ele está na pasta certa.

Estrutura — biblioteca_projeto/ (estrutura final)

```
biblioteca_projeto/
├── manage.py
├── requirements.txt
├── .gitignore
├── README.md
├── config/
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── biblioteca/
    ├── migrations/
    ├── templates/
    │   └── biblioteca/
    │       ├── base.html
    │       ├── lista_livros.html
    │       ├── meus_emprestimos.html
    │       └── relatorio_livros.html
    ├── models.py
    ├── admin.py
    ├── views.py
    ├── urls.py
    └── tests.py
```

> **Atenção à pasta dupla de templates**
>
> Os templates ficam em `biblioteca/templates/biblioteca/`. A pasta repetida é proposital: evita conflito de nomes entre apps. Se o Django disser `TemplateDoesNotExist`, confira primeiro este caminho.

3. Passo a passo

1 Preparar o ambiente

Sempre o primeiro passo: pasta, ambiente virtual, Django

Abra o terminal do VS Code (Ctrl + ') na pasta onde ficam seus projetos e execute, um comando por vez:

```
$ mkdir biblioteca_projeto
$ cd biblioteca_projeto
$ python -m venv venv
$ venv\Scripts\activate
$ pip install django
$ django-admin startproject config .
```

No Linux/macOS, ative o ambiente com `source venv/bin/activate`. O **ponto final** em `startproject config .` é obrigatório: ele cria o projeto na pasta atual, sem uma pasta extra.

> **Banco de dados**
>
> Use o **mesmo banco que você configurou nas Aulas 05 e 06** (Supabase/PostgreSQL com variáveis no arquivo `.env`, lidas com `python-dotenv`). Se preferir testar rápido, o SQLite padrão do Django também funciona: este projeto não depende do SGBD escolhido.
>
> Lembre: **nunca** coloque senha de banco direto no `settings.py`, e nunca envie o `.env` ao GitHub.

Checkpoint 1: rode `python manage.py runserver` e abra `http://127.0.0.1:8000`. Deve aparecer a página do foguete do Django. Encerre o servidor com Ctrl + C.

2 Criar o app e registrá-lo

O app biblioteca guarda os Models, Views e templates

```
$ python manage.py startapp biblioteca
```

Abra `config/settings.py` e acrescente `"biblioteca"` à lista `INSTALLED_APPS` (mantenha os apps que já existem):

Código 1 — config/settings.py (trecho — mantenha o restante do arquivo)

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "biblioteca",
]
```

Checkpoint 2: `python manage.py check` deve responder `System check identified no issues`.

3 Criar os 4 Models

Autor, Livro, Membro e Emprestimo

Substitua todo o conteúdo de `biblioteca/models.py` pelo código abaixo.

Código 2 — biblioteca/models.py (arquivo completo)

```
from django.db import models
from django.utils import timezone


class Autor(models.Model):
    nome = models.CharField(max_length=150)
    nacionalidade = models.CharField(max_length=80)
    data_nascimento = models.DateField(null=True, blank=True)
```

```
    def __str__(self):
        return self.nome


class Livro(models.Model):
    titulo = models.CharField(max_length=200)
    autor = models.ForeignKey(Autor, on_delete=models.CASCADE, related_name='livros')
    isbn = models.CharField(max_length=20, unique=True)
    copias_totais = models.PositiveIntegerField()
    copias_disponiveis = models.PositiveIntegerField()

    def __str__(self):
        return self.titulo


class Membro(models.Model):
    nome = models.CharField(max_length=150)
    email = models.EmailField(unique=True)
    data_cadastro = models.DateField(auto_now_add=True)

    def __str__(self):
        return self.nome


class Emprestimo(models.Model):
    STATUS_CHOICES = [
        ('ATIVO', 'Ativo'),
        ('DEVOLVIDO', 'Devolvido'),
        ('ATRASADO', 'Atrasado'),
    ]

    livro = models.ForeignKey(Livro, on_delete=models.CASCADE)
    membro = models.ForeignKey(Membro, on_delete=models.CASCADE)
    data_emprestimo = models.DateField(auto_now_add=True)
    data_prevista_devolucao = models.DateField()
    data_devolucao = models.DateField(null=True, blank=True)
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='ATIVO')

    @property
    def esta_atrasado(self):
        return self.status == 'ATIVO' and self.data_prevista_devolucao < timezone.localdate()

    def __str__(self):
        return f'{self.membro} -> {self.livro}'
```

Entenda antes de seguir

- `ForeignKey(Autor, ..., related_name='livros')`: cada Livro pertence a **um** Autor; e de volta, `autor.livros.all()` lista os livros dele.
- `unique=True` em `isbn` e `email`: o banco recusa duplicados.
- `auto_now_add=True`: o Django preenche a data sozinho, na criação.
- `esta_atrasado` é uma **property**: calculada na hora, **não** vira coluna. Só está atrasado quem ainda está `ATIVO` e passou do prazo.
- `timezone.localdate()` devolve a data de hoje no fuso configurado no projeto, evitando erro de um dia à noite.

4 Gerar e aplicar as migrations

Transformar os Models em tabelas

```
$ python manage.py makemigrations biblioteca
$ python manage.py migrate
```

Checkpoint 4: o primeiro comando deve listar `Create model Autor`, `Membro`, `Livro` e `Emprestimo`. O segundo deve terminar com `OK`.

5 Configurar o Django Admin

list_display, filtros e TabularInline

Substitua o conteúdo de `biblioteca/admin.py`:

Código 3 — biblioteca/admin.py (arquivo completo)

```python
from django.contrib import admin
from .models import Autor, Livro, Membro, Emprestimo


@admin.register(Autor)
class AutorAdmin(admin.ModelAdmin):
    list_display = ('nome', 'nacionalidade', 'data_nascimento')
    search_fields = ('nome',)


@admin.register(Livro)
class LivroAdmin(admin.ModelAdmin):
    list_display = ('titulo', 'autor', 'copias_totais', 'copias_disponiveis')
    list_filter = ('autor',)
    search_fields = ('titulo', 'isbn')


class EmprestimoInline(admin.TabularInline):
    model = Emprestimo
    extra = 0
    fields = ('livro', 'data_prevista_devolucao', 'data_devolucao', 'status')


@admin.register(Membro)
class MembroAdmin(admin.ModelAdmin):
    list_display = ('nome', 'email', 'data_cadastro')
    inlines = [EmprestimoInline]
```

Crie seu usuário administrador e suba o servidor:

```
$ python manage.py createsuperuser
$ python manage.py runserver
```

Acesse `http://127.0.0.1:8000/admin/`, entre e **cadastre os dados de teste**: no mínimo **3 Autores, 5 Livros e 3 Membros**. Em cada Livro, coloque `copias_totais` e `copias_disponiveis` com o mesmo valor (ex.: 2 e 2). Deixe **um** livro com `0` cópias disponíveis para testar o bloqueio.

> **Cuidado ao cadastrar Empréstimos pelo Admin**
> O Admin cria o registro, mas **não** aplica as regras de negócio (não diminui o estoque). Empréstimos de verdade devem ser feitos pela **View** do Passo 7. Use o `TabularInline` apenas para **consultar** os empréstimos do Membro.

Checkpoint 5: em Membros, abrir um Membro e ver a tabela de empréstimos dentro da mesma tela; em Livros, ver as colunas Autor e Cópias.

6 Criar o template base

Menu e área de mensagens compartilhados

Crie as pastas `biblioteca/templates/biblioteca/` e, dentro delas, o arquivo `base.html`. As **mensagens** (sucesso, erro, aviso) das Views aparecem aqui.

Código 4 — biblioteca/templates/biblioteca/base.html (arquivo completo)

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block titulo %}Biblioteca IESB{% endblock %}</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 900px; margin: 2rem auto; padding: 0
1rem; }
        nav a { margin-right: 1rem; }
        table { border-collapse: collapse; width: 100%; }
        th, td { border: 1px solid #ccc; padding: 6px 10px; text-align: left; }
        .msg { padding: 8px 12px; margin: 8px 0; border-radius: 4px; }
        .msg.success { background: #d4edda; }
        .msg.error { background: #f8d7da; }
        .msg.warning { background: #fff3cd; }
    </style>
</head>
<body>
    <nav>
        <a href="{% url 'lista_livros' %}">Livros</a>
        <a href="{% url 'relatorio_livros' %}">Relatório</a>
        <a href="/admin/">Admin</a>
    </nav>
    <hr>
    {% for message in messages %}
        <div class="msg {{ message.tags }}">{{ message }}</div>
    {% endfor %}
    {% block conteudo %}{% endblock %}
</body>
</html>
```

7 Escrever as Views com as regras de negócio

O coração do projeto

Substitua o conteúdo de `biblioteca/views.py`. Leia os comentários: eles indicam cada regra.

Código 5 — biblioteca/views.py (arquivo completo)

```python
from datetime import timedelta

from django.contrib import messages
from django.db.models import Count
from django.shortcuts import get_object_or_404, redirect, render
from django.utils import timezone
from django.views.decorators.http import require_POST

from .models import Emprestimo, Livro, Membro


def lista_livros(request):
    livros = Livro.objects.select_related('autor').order_by('titulo')
    membros = Membro.objects.order_by('nome')
    return render(request, 'biblioteca/lista_livros.html', {
        'livros': livros,
        'membros': membros,
    })


@require_POST
def emprestar_livro(request, livro_id):
    livro = get_object_or_404(Livro, pk=livro_id)

    # Regra de negócio 1: sem cópias disponíveis, não empresta.
    if livro.copias_disponiveis <= 0:
        messages.error(request, f'Não há cópias disponíveis de "{livro.titulo}" no momento.')
        return redirect('lista_livros')

    membro = get_object_or_404(Membro, pk=request.POST.get('membro_id'))

    # Regra de negócio 2: prazo de 14 dias a partir de hoje.
    Emprestimo.objects.create(
        livro=livro,
        membro=membro,
        data_prevista_devolucao=timezone.localdate() + timedelta(days=14),
    )
    livro.copias_disponiveis -= 1
    livro.save()

    messages.success(request, f'Empréstimo de "{livro.titulo}" registrado com sucesso!')
    return redirect('lista_livros')


@require_POST
def devolver_livro(request, emprestimo_id):
    emprestimo = get_object_or_404(Emprestimo, pk=emprestimo_id)

    # Proteção: devolver duas vezes inflaria o estoque.
    if emprestimo.status == 'DEVOLVIDO':
        messages.warning(request, 'Este empréstimo já foi devolvido.')
```

```
        return redirect('meus_emprestimos', membro_id=emprestimo.membro.id)

    emprestimo.data_devolucao = timezone.localdate()
    emprestimo.status = 'DEVOLVIDO'
    emprestimo.save()

    emprestimo.livro.copias_disponiveis += 1
    emprestimo.livro.save()

    messages.success(request, f'Devolução de "{emprestimo.livro.titulo}" registrada.')
    return redirect('meus_emprestimos', membro_id=emprestimo.membro.id)


def meus_emprestimos(request, membro_id):
    membro = get_object_or_404(Membro, pk=membro_id)
    emprestimos = membro.emprestimo_set.select_related('livro').order_by('-data_emprestimo')
    return render(request, 'biblioteca/meus_emprestimos.html', {
        'membro': membro,
        'emprestimos': emprestimos,
    })


def relatorio_livros(request):
    livros = Livro.objects.annotate(
        total_emprestimos=Count('emprestimo')
    ).order_by('-total_emprestimos', 'titulo')
    return render(request, 'biblioteca/relatorio_livros.html', {'livros': livros})
```

O que cada View faz

View	Responsabilidade
`lista_livros`	Mostra o acervo e um formulário para emprestar cada livro a um Membro.
`emprestar_livro`	Valida o estoque (regra 1), cria o Emprestimo com prazo de 14 dias (regra 2) e diminui `copias_disponiveis`.
`devolver_livro`	Registra a data, muda o status para `DEVOLVIDO` e devolve a cópia ao estoque. Impede devolver duas vezes.
`meus_emprestimos`	Lista os empréstimos de um Membro via relação reversa `membro.emprestimo_set`.
`relatorio_livros`	`annotate(Count('emprestimo'))` e `order_by('-total_emprestimos')`: ranking dos mais emprestados.

> **Por que @require_POST?**
> Emprestar e devolver **alteram dados**. Alteração nunca deve acontecer por um simples link (GET), senão um clique acidental, ou um robô, mudaria o estoque. Por isso os botões usam formulário com `{% csrf_token %}`.

8 Configurar as URLs

Ligar endereços às Views

Código 6 — biblioteca/urls.py (arquivo completo)

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.lista_livros, name='lista_livros'),
    path('emprestar/<int:livro_id>/', views.emprestar_livro, name='emprestar_livro'),
    path('devolver/<int:emprestimo_id>/', views.devolver_livro, name='devolver_livro'),
    path('membro/<int:membro_id>/emprestimos/', views.meus_emprestimos,
name='meus_emprestimos'),
    path('relatorio/', views.relatorio_livros, name='relatorio_livros'),
]
```

Agora conecte o app ao projeto. Substitua `config/urls.py`:

Código 7 — config/urls.py (arquivo completo)

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("biblioteca.urls")),
]
```

9 Criar os templates das telas

Acervo, empréstimos do membro e relatório

Código 8 — biblioteca/templates/biblioteca/lista_livros.html (arquivo completo)

```
{% extends 'biblioteca/base.html' %}

{% block titulo %}Livros | Biblioteca IESB{% endblock %}

{% block conteudo %}
<h1>Acervo de Livros</h1>
<table>
    <tr>
        <th>Título</th>
        <th>Autor</th>
        <th>Disponíveis</th>
        <th>Emprestar para</th>
    </tr>
    {% for livro in livros %}
    <tr>
        <td>{{ livro.titulo }}</td>
        <td>{{ livro.autor.nome }}</td>
        <td>{{ livro.copias_disponiveis }} / {{ livro.copias_totais }}</td>
        <td>
            <form method="post" action="{% url 'emprestar_livro' livro.id %}">
```

```
                {% csrf_token %}
                <select name="membro_id" required>
                    {% for membro in membros %}
                    <option value="{{ membro.id }}">{{ membro.nome }}</option>
                    {% endfor %}
                </select>
                <button type="submit">Emprestar</button>
            </form>
        </td>
    </tr>
    {% empty %}
    <tr><td colspan="4">Nenhum livro cadastrado. Use o Admin para cadastrar.</td></tr>
    {% endfor %}
</table>

<h2>Empréstimos por membro</h2>
<ul>
    {% for membro in membros %}
    <li><a href="{% url 'meus_emprestimos' membro.id %}">{{ membro.nome }}</a></li>
    {% endfor %}
</ul>
{% endblock %}
```

Código 9 — biblioteca/templates/biblioteca/meus_emprestimos.html (arquivo completo)

```
{% extends 'biblioteca/base.html' %}

{% block titulo %}Empréstimos de {{ membro.nome }}{% endblock %}

{% block conteudo %}
<h1>Empréstimos de {{ membro.nome }}</h1>
<table>
    <tr>
        <th>Livro</th>
        <th>Devolução prevista</th>
        <th>Situação</th>
        <th>Ação</th>
    </tr>
    {% for emprestimo in emprestimos %}
    <tr>
        <td>{{ emprestimo.livro.titulo }}</td>
        <td>{{ emprestimo.data_prevista_devolucao|date:"d/m/Y" }}</td>
        <td>
            {% if emprestimo.esta_atrasado %}
                <span style="color:red">Atrasado</span>
            {% elif emprestimo.status == 'DEVOLVIDO' %}
                <span style="color:green">Devolvido</span>
            {% else %}
                <span style="color:orange">Em andamento</span>
            {% endif %}
        </td>
        <td>
            {% if emprestimo.status != 'DEVOLVIDO' %}
            <form method="post" action="{% url 'devolver_livro' emprestimo.id %}">
                {% csrf_token %}
                <button type="submit">Devolver</button>
            </form>
            {% endif %}
        </td>
    </tr>
    {% empty %}
    <tr><td colspan="4">Este membro ainda não tem empréstimos.</td></tr>
```

```
    {% endfor %}
</table>
<p><a href="{% url 'lista_livros' %}">&larr; Voltar ao acervo</a></p>
{% endblock %}
```

Código 10 — biblioteca/templates/biblioteca/relatorio_livros.html (arquivo completo)

```
{% extends 'biblioteca/base.html' %}

{% block titulo %}Relatório | Biblioteca IESB{% endblock %}

{% block conteudo %}
<h1>Livros mais emprestados</h1>
<table>
    <tr>
        <th>#</th>
        <th>Título</th>
        <th>Autor</th>
        <th>Total de empréstimos</th>
    </tr>
    {% for livro in livros %}
    <tr>
        <td>{{ forloop.counter }}</td>
        <td>{{ livro.titulo }}</td>
        <td>{{ livro.autor.nome }}</td>
        <td>{{ livro.total_emprestimos }}</td>
    </tr>
    {% empty %}
    <tr><td colspan="4">Sem dados.</td></tr>
    {% endfor %}
</table>
{% endblock %}
```

No `meus_emprestimos.html`, o **badge de atraso** usa a property: `{% if emprestimo.esta_atrasado %}`. Note que o template chama `esta_atrasado` **sem parênteses**, pois é uma property.

10 Testar os fluxos na mão

Roteiro de teste — marque cada linha

Com o servidor rodando, abra `http://127.0.0.1:8000/` e execute a tabela abaixo:

#	Ação	Resultado esperado	OK
1	Emprestar um livro com cópias para um Membro	Mensagem verde de sucesso; disponíveis diminuem 1	☐
2	Tentar emprestar o livro com 0 cópias	Mensagem vermelha; nada é criado	☐
3	Abrir os empréstimos do Membro	Empréstimo aparece como "Em andamento", com prazo daqui a 14 dias	☐
4	Clicar em Devolver	Status "Devolvido"; disponíveis voltam a subir 1	☐
5	Atualizar a página e reenviar a devolução	O estoque não aumenta de novo	☐

#	Ação	Resultado esperado	OK
6	No Admin, editar um empréstimo e voltar o prazo para uma data passada (status Ativo)	Badge vermelho "Atrasado" em "meus empréstimos"	☐
7	Abrir `/relatorio/`	Livros ordenados do mais para o menos emprestado	☐

11 Explorar o ORM no shell

related_name, relação reversa e annotate

```
$ python manage.py shell
```

Código 11 — Terminal do shell do Django (digite linha a linha)

```
from biblioteca.models import Autor, Livro, Membro
from django.db.models import Count

# 1) related_name: livros de um autor, sem consulta manual
autor = Autor.objects.first()
autor.livros.all()

# 2) relação reversa do Emprestimo (sem related_name definido)
membro = Membro.objects.first()
membro.emprestimo_set.all()

# 3) agregação: livros mais emprestados
ranking = Livro.objects.annotate(total_emprestimos=Count('emprestimo')).order_by('-
total_emprestimos')
for livro in ranking:
    print(livro.titulo, livro.total_emprestimos)
```

Saia com `exit()`. **Checkpoint 11:** o ranking deve imprimir os títulos do maior para o menor número de empréstimos.

12 Rodar os testes automatizados

Provar que as regras funcionam

Substitua `biblioteca/tests.py` e execute `python manage.py test`. Cada teste verifica uma regra do enunciado. Se algum falhar, a mensagem indica qual regra você quebrou.

Código 12 — biblioteca/tests.py (arquivo completo)

```
from datetime import timedelta

from django.db.models import Count
from django.test import TestCase
from django.urls import reverse
from django.utils import timezone

from .models import Autor, Emprestimo, Livro, Membro
```

```
class RegrasDeNegocioTest(TestCase):
    def setUp(self):
        self.autor = Autor.objects.create(nome='Machado de Assis', nacionalidade='Brasileira')
        self.livro = Livro.objects.create(
            titulo='Dom Casmurro', autor=self.autor, isbn='111', copias_totais=1,
copias_disponiveis=1)
        self.sem_copia = Livro.objects.create(
            titulo='O Cortico', autor=self.autor, isbn='222', copias_totais=1,
copias_disponiveis=0)
        self.membro = Membro.objects.create(nome='Ana', email='ana@iesb.br')

    def test_emprestar_define_prazo_e_decrementa(self):
        r = self.client.post(reverse('emprestar_livro', args=[self.livro.id]), {'membro_id':
self.membro.id})
        self.assertEqual(r.status_code, 302)
        emp = Emprestimo.objects.get()
        self.assertEqual(emp.data_prevista_devolucao, timezone.localdate() +
timedelta(days=14))
        self.assertEqual(emp.status, 'ATIVO')
        self.livro.refresh_from_db()
        self.assertEqual(self.livro.copias_disponiveis, 0)

    def test_bloqueia_sem_copias(self):
        self.client.post(reverse('emprestar_livro', args=[self.sem_copia.id]), {'membro_id':
self.membro.id})
        self.assertEqual(Emprestimo.objects.count(), 0)
        self.sem_copia.refresh_from_db()
        self.assertEqual(self.sem_copia.copias_disponiveis, 0)

    def test_get_nao_emprestar(self):
        r = self.client.get(reverse('emprestar_livro', args=[self.livro.id]))
        self.assertEqual(r.status_code, 405)

    def test_devolver_repoe_estoque_e_nao_duplica(self):
        self.client.post(reverse('emprestar_livro', args=[self.livro.id]), {'membro_id':
self.membro.id})
        emp = Emprestimo.objects.get()
        self.client.post(reverse('devolver_livro', args=[emp.id]))
        self.client.post(reverse('devolver_livro', args=[emp.id]))  # segunda vez: ignorada
        emp.refresh_from_db()
        self.livro.refresh_from_db()
        self.assertEqual(emp.status, 'DEVOLVIDO')
        self.assertEqual(emp.data_devolucao, timezone.localdate())
        self.assertEqual(self.livro.copias_disponiveis, 1)

    def test_esta_atrasado(self):
        hoje = timezone.localdate()
        atrasado = Emprestimo.objects.create(
            livro=self.livro, membro=self.membro, data_prevista_devolucao=hoje -
timedelta(days=1))
        no_prazo = Emprestimo.objects.create(
            livro=self.livro, membro=self.membro, data_prevista_devolucao=hoje +
timedelta(days=3))
        devolvido = Emprestimo.objects.create(
            livro=self.livro, membro=self.membro, data_prevista_devolucao=hoje -
timedelta(days=5),
            status='DEVOLVIDO')
        self.assertTrue(atrasado.esta_atrasado)
        self.assertFalse(no_prazo.esta_atrasado)
        self.assertFalse(devolvido.esta_atrasado)
```

```
    def test_meus_emprestimos_mostra_badge(self):
        Emprestimo.objects.create(
            livro=self.livro, membro=self.membro,
            data_prevista_devolucao=timezone.localdate() - timedelta(days=2))
        r = self.client.get(reverse('meus_emprestimos', args=[self.membro.id]))
        self.assertContains(r, 'Atrasado')

    def test_related_name_e_ranking(self):
        self.assertEqual(self.autor.livros.count(), 2)
        for _ in range(3):
            Emprestimo.objects.create(livro=self.livro, membro=self.membro,
                                      data_prevista_devolucao=timezone.localdate())
        Emprestimo.objects.create(livro=self.sem_copia, membro=self.membro,
                                  data_prevista_devolucao=timezone.localdate())
        ranking = list(Livro.objects.annotate(total=Count('emprestimo')).order_by('-total'))
        self.assertEqual(ranking[0].titulo, 'Dom Casmurro')
        self.assertEqual(ranking[0].total, 3)
        self.assertEqual(self.client.get(reverse('relatorio_livros')).status_code, 200)
        self.assertEqual(self.client.get(reverse('lista_livros')).status_code, 200)

    def test_admin_registrado(self):
        from django.contrib import admin
        for m in (Autor, Livro, Membro):
            self.assertIn(m, admin.site._registry)
```

```
$ python manage.py test
```

Checkpoint 12: a saída termina com `Ran 8 tests` e `OK`.

13 Publicar no GitHub

requirements.txt, .gitignore e README

```
$ pip freeze > requirements.txt
```

Código 13 — .gitignore (arquivo completo)

```
venv/
__pycache__/
*.pyc
db.sqlite3
.env
.vscode/
```

Código 14 — README.md (modelo — preencha com os seus dados)

```
# Sistema de Biblioteca — Projeto 2

Projeto da disciplina Construção de Backend (IESB). Gerencia autores,
livros, membros e empréstimos com regras de negócio em Django.

## Funcionalidades
- Cadastro de Autores, Livros e Membros pelo Django Admin
- Empréstimo com prazo de 14 dias e controle de estoque
```

```
- Devolução com reposição de cópia
- Indicação de empréstimos atrasados
- Relatório de livros mais emprestados

## Como rodar
1. python -m venv venv  e ative o ambiente
2. pip install -r requirements.txt
3. python manage.py migrate
4. python manage.py createsuperuser
5. python manage.py runserver

## Prints
(inclua prints do Admin, da listagem de livros e dos empréstimos)

## Autor
Seu nome — IESB 2026
```

```
$ git init
$ git add .
$ git commit -m "Projeto 2: Sistema de Biblioteca"
$ git branch -M main
$ git remote add origin <URL-DO-SEU-REPOSITORIO>
$ git push -u origin main
```

> **Antes do push, confira**
>
> O repositório deve ser **público**, com `README.md` e prints das telas.
>
> `venv/`, `db.sqlite3` e `.env` **não** podem aparecer no GitHub. Se aparecerem, o `.gitignore` foi criado tarde demais: peça ajuda ao professor antes de continuar.

4. Checklist de entrega

Item	Feito
Os 4 Models (`Autor`, `Livro`, `Membro`, `Emprestimo`) criados e migrados sem erros	☐
Admin com `AutorAdmin`, `LivroAdmin` (autor e `copias_disponiveis` no `list_display`) e `MembroAdmin` com `EmprestimoInline`	☐
A View de empréstimo bloqueia quando `copias_disponiveis` é zero, sem quebrar a aplicação	☐
A View de devolução incrementa `copias_disponiveis` e atualiza o `status`	☐
O template "meus empréstimos" exibe o badge de atraso usando `esta_atrasado`	☐
A consulta de "mais emprestados" com `annotate()` + `Count()` funciona no shell	☐
Banco populado com no mínimo 3 Autores, 5 Livros e 3 Membros	☐
`python manage.py test` passa	☐
Repositório público no GitHub com `README.md`, prints, `requirements.txt` e `.gitignore`	☐

5. Erros comuns e como resolver

Mensagem / sintoma	Causa provável	Como resolver
`TemplateDoesNotExist`	Pasta de templates no lugar errado	Conferir `biblioteca/templates/biblioteca/` e se `"biblioteca"` está em `INSTALLED_APPS`
`NoReverseMatch`	Nome da URL diferente do usado em `{% url %}` ou `redirect`	Comparar o `name=` em `urls.py` com o nome usado
`Method Not Allowed (405)` ao emprestar	Acesso por link (GET) a uma View `@require_POST`	Usar o botão do formulário, não digitar o endereço
`CSRF verification failed`	Formulário sem `{% csrf_token %}`	Adicionar o token logo após a tag `<form>`
`IntegrityError` em ISBN ou e-mail	Valor duplicado (`unique=True`)	Usar outro ISBN/e-mail no cadastro
`no such table`	Migrations não aplicadas	`makemigrations biblioteca` e depois `migrate`
Badge de atraso não aparece	Empréstimo já devolvido, ou prazo ainda no futuro	Só empréstimos `ATIVO` com prazo vencido ficam atrasados
Estoque negativo ou maior que o total	Cadastro manual inconsistente no Admin	Manter `copias_disponiveis` entre 0 e `copias_totais`

6. Desafios extras (opcionais)

Quem terminar antes pode escolher um ou mais desafios. Eles valem destaque no portfólio.

Desafio	O que fazer	Dica
Multa por atraso	Calcular R$ por dia de atraso (data prevista até a devolução, ou até hoje se ainda não devolvido)	Duas properties no Model: dias de atraso e multa. Use `Decimal` para dinheiro
Reserva de livro	Fila de reserva quando todas as cópias estiverem emprestadas	Crie um Model `Reserva` ligado a Livro e Membro, com data e ordem
Notificação por e-mail	Avisar o Membro 2 dias antes do prazo	`send_mail` do Django; teste com o backend de console
Limite de empréstimos	No máximo 3 empréstimos `ATIVO` por Membro	`Emprestimo.objects.filter(membro=..., status='ATIVO').count()` antes de criar
Dashboard	Gráfico de empréstimos por mês	Agrupe com o ORM e use Chart.js

> **Bom trabalho!**
> Quando terminar, volte ao checklist, revise o repositório e envie o link do GitHub conforme combinado com o professor.